

DATA STRUCTURES AND ALGORITHMS I

Assignment Report

Question 1: Arrays – University Course Registration Analytics

Introduction

The objective of this task was to implement efficient algorithms for analyzing student course registration data stored in a Python list. The program performs several operations, including identifying the most frequent student ID, removing duplicate registrations while preserving the original order, finding the first student who registered only once, and determining whether a contiguous subarray sums to a specified target value. Additionally, the time and space complexities of each algorithm were analyzed using Big-O notation.

Methodology

Hash tables (Python dictionaries) were used to count the frequency of student IDs, allowing the program to efficiently identify the most frequent and first unique student. A hash set was used to remove duplicate entries while maintaining the order of first appearance. For the contiguous subarray sum problem, the prefix sum technique combined with a hash set was implemented to achieve efficient searching. These approaches ensure that each operation runs efficiently even for large datasets.

Results

The program successfully identified the most frequent student ID, produced a list of unique registrations in their original order, located the first non-repeated student, and correctly determined whether a contiguous subarray with the specified target sum exists.

Complexity Analysis

All four algorithms execute in $O(n)$ time, where n is the number of registrations. They also require $O(n)$ additional memory because auxiliary data structures such as dictionaries and sets are used to improve performance.

Conclusion

The implementation demonstrates how appropriate use of hash-based data structures can significantly improve the efficiency of array operations. The algorithms are scalable and suitable for processing large registration datasets.

Question 2: Linked Lists – University Helpdesk Ticket Queue

Introduction

The purpose of this task was to implement a singly linked list to manage helpdesk support tickets. Unlike arrays, linked lists provide flexible memory allocation and efficient insertion or deletion of elements without requiring data shifting.

Methodology

A Ticket class was created to represent each support ticket, while a TicketQueue class managed the linked list operations. The implemented operations include enqueueing new tickets, inserting priority tickets after a specified ticket, resolving tickets by deletion, finding the middle ticket using the fast and slow pointer technique, reversing the queue in-place, and displaying all ticket details.

Results

The linked list successfully maintained the order of support tickets while supporting all required operations. The fast and slow pointer algorithm efficiently located the middle ticket in a single traversal, and pointer manipulation enabled the queue to be reversed without copying data.

Complexity Analysis

Most operations require traversing the linked list and therefore have a worst-case time complexity of $O(n)$. The implementation requires only $O(1)$ additional space since operations manipulate existing nodes without creating duplicate structures.

Conclusion

The singly linked list is an effective data structure for applications involving frequent insertions and deletions. Its dynamic memory allocation and pointer-based implementation make it well suited for managing continuously changing helpdesk ticket queues.

Question 3: Bonus Challenge

Introduction

This task required evaluating whether a Python list would be a better choice than a singly linked list if the university helpdesk performs thousands of search operations daily but only a few insertions and deletions.

Discussion

Although searching in both Python lists and singly linked lists has a theoretical worst-case complexity of $O(n)$, Python lists generally provide much faster practical performance. This is because list elements are stored contiguously in memory, allowing the processor to utilize CPU cache effectively. Sequential memory access minimizes cache misses and improves execution speed.

In contrast, linked lists store nodes in separate memory locations connected by pointers. Traversing the list requires following these pointers, resulting in slower memory access and reduced cache efficiency. Linked lists also consume additional memory because each node stores a pointer alongside the actual data.

Conclusion

For workloads dominated by search operations, a Python list is the preferred data structure due to its superior cache locality, lower memory overhead, and faster practical performance. A singly linked list remains advantageous only when applications require frequent insertions and deletions, particularly in the middle of the data structure.